

DD_PAY-GW-01 - Payment Gateway Integration

Version: 1.0 Status: Draft for review Bundle: 05_billing Audience: backend developers, billing developers, integration developers, QA, DevOps, operations Related DDs:

- DD_BIL-01-PAY-01 - Payment Application
- DD_BIL-05 - Wallet and Topup
- DD_BIL-02-TAX-01 - Tax Invoice and Gateway
- DD_API-00 - API Catalog and Gateway Standards
- DD_EM-CFG-03 - RBAC Catalog
- FE-APP-01 - BSS Backoffice Web App
- FE-APP-02 - Outdoor Sales Franchise App
- FE-APP-04 - Customer Self Care Frontoffice App

MVP simplification baseline: first release implements one selected Kenya payment provider class end to end, including callback idempotency, payment routing to BIL, settlement import, and manual repair. Additional providers/channels are configuration-compatible but phased after the first provider is stable.

1. Purpose

PAY-GW-01 defines how SOPHIX receives, validates, stores, reconciles, and routes payments coming from external payment providers.

This includes mobile money, bank transfer notifications, card processors, retail-agent payment systems, and future regional payment channels.

For the Kenya-first deployment, the first provider class is expected to be M-Pesa-style payment callbacks, with bank transfer and card channels added as operator configuration requires. The design must remain configurable enough to support other regions with different mobile-money or bank providers.

This DD exists because payment gateway integration is not the same thing as payment application:

Concern	Owning module
External provider callback URL, signature check, raw payload audit, provider idempotency	PAY-GW-01
Deciding whether money pays an invoice, a customer account, or wallet topup	PAY-GW-01 routing rules, then target module
Applying a POSTPAID payment to invoice/customer balance	BIL-01-PAY-01
Crediting a PREPAID wallet	BIL-05
Tax invoice trigger after payment or topup	BIL-02-TAX-01 consumes BIL-01/BIL-05 events
Displaying payment options to customers or sales users	FE apps read PAY-GW channel config
Nightly provider settlement comparison and payment repair queue	PAY-GW-01

2. Scope

2.1 In Scope

- Payment provider catalog and channel configuration.
- Public provider callback endpoints.
- Provider payload validation and signature verification.
- Raw callback audit with sensitive data redaction.
- Gateway transaction registry.
- Gateway idempotency and duplicate callback handling.
- Routing a validated payment to:

- BIL - 01 - PAY - 01 for invoice/account payment application.
- BIL - 05 for PREPAID wallet topup.
- Held payment queue for payments that cannot be resolved automatically.
- Manual repair actions from Backoffice.
- Provider settlement import or API reconciliation.
- Reconciliation discrepancy queue.
- Gateway events for observability and downstream audit.
- Infra and AWS deployment recommendations for gateway-facing services.

2.2 Out of Scope

- Invoice allocation rules. These belong to BIL - 01 - PAY - 01.
- Wallet balance ledger rules. These belong to BIL - 05.
- Dunning behavior after payment. This belongs to BIL - 04.
- Tax invoice signing and tax authority integration. This belongs to BIL - 02 - TAX - 01.
- Outbound payout execution for refunds. This is a separate payout capability. PAY-GW-01 may expose shared provider configuration, but does not execute refund payouts in this DD.
- Customer-facing UI implementation. FE apps consume the APIs defined here.

3. Design Decision

PAY-GW-01 is the integration boundary between external payment providers and the internal billing modules.

The module must first persist the provider callback durably, then route it asynchronously to the correct internal target. This prevents provider retry behavior, network latency, or internal billing outages from losing payments.

Preferred callback ownership:

```
External provider
-> PAY-GW public callback endpoint
-> gateway_callback_event
-> gateway_transaction
-> routing worker
-> BIL-01-PAY-01 or BIL-05
```

BIL-05 currently mentions gateway-specific callback URLs for wallet topups. Those URLs should be treated as a legacy or compatibility shape. New implementation should expose provider callbacks under PAY-GW-01 and route wallet topups into BIL-05 using internal authenticated APIs.

Compatibility wrappers may remain if already implemented:

Legacy shape	Preferred shape
POST /api/gateway/mpesa/callback in BIL-05	POST /api/payment-gateways/mpesa/callback in PAY-GW-01
GET /api/gateway-reconciliations in BIL-05	GET /api/payment-gateways/reconciliation-runs in PAY-GW-01

4. High Level Architecture

```
flowchart LR
    Provider["Payment Provider<br/>M-Pesa, bank, card, retail"]
    PublicAPI["PAY-GW Public Callback API"]
    Registry["Gateway Transaction Registry"]
    Router["Routing Worker"]
    Pay01["BIL-01-PAY-01<br/>Payment Application"]
    Wallet["BIL-05<br/>Wallet and Topup"]
    Recon["Settlement Reconciliation Worker"]
    Backoffice["FE-APP-01<br/>Payment Ops Console"]
    Kafka["Kafka Events"]

    Provider --> PublicAPI
    PublicAPI --> Registry
    Registry --> Router
    Router --> Pay01
    Router --> Wallet
    Router --> Recon
    Router --> Backoffice
    Router --> Kafka
```

```
Router --> Pay01
Router --> Wallet
Registry --> Recon
Recon --> Registry
Backoffice --> Registry
Registry --> Kafka
```

5. Core Concepts

5.1 Provider

A provider is an external payment system integrated with SOPHIX.

Examples:

- MPESA_KE
- BANK_TRANSFER_KE
- CARD_PROCESSOR_KE
- RETAIL_AGENT_KE
- Future region-specific mobile-money providers

5.2 Channel

A channel is a customer-visible or operational payment route under a provider.

Examples:

- M-Pesa Paybill for customer invoice payment.
- M-Pesa STK Push for self-care payment.
- Bank transfer to operator account.
- Retail-agent cash collection.

5.3 Callback Event

A callback event is the exact inbound message received from the provider. It is stored for audit and idempotency, with sensitive fields redacted or encrypted.

5.4 Gateway Transaction

A gateway transaction is SOPHIX's normalized representation of money received or reversed through an external provider.

One provider callback may produce one gateway transaction. For batch settlement files, one settlement row may also produce or update one gateway transaction.

5.5 Routing

Routing decides which internal module must receive the money:

Target	Route to
Customer invoice or account payment	BIL - 01 - PAY - 01
PREPAID wallet topup	BIL - 05
Unknown customer/reference	Hold in PAY-GW queue
Suspected duplicate conflict	Hold in PAY-GW queue

6. Payment Flow

6.1 Provider Callback Happy Path

```
sequenceDiagram
    participant P as Payment Provider
    participant G as PAY-GW API
```

```

participant DB as PAY-GW DB
participant W as PAY-GW Router
participant B as BIL-01 or BIL-05
participant E as Kafka

P->>G: POST callback
G->>G: Verify signature and parse payload
G->>DB: Insert callback event
G->>DB: Upsert gateway transaction
G-->>P: 200 accepted after durable store
W->>DB: Load unrouted transaction
W->>B: Apply payment or credit wallet
B-->>W: Success with internal reference
W->>DB: Mark APPLIED
W->>E: GatewayPaymentApplied

```

6.2 Important Callback Rule

PAY-GW returns success to the provider only after the callback is durably stored.

Routing to BIL-01 or BIL-05 should normally happen asynchronously, because provider callbacks often have short timeout windows. If internal routing fails after storage, the routing worker retries and operations can repair the transaction from Backoffice.

6.3 Duplicate Callback Rule

If the same provider transaction arrives again with the same normalized provider transaction id and same amount/currency/payment date, PAY-GW must return 200 and not route a second payment.

If the same provider transaction id arrives with a different amount, currency, payer, or paid date, PAY-GW must hold the transaction as DUPLICATE_CONFLICT and require operations review.

7. Payment Routing Rules

7.1 Route To Payment Application

Use this route when the payment is meant to pay a POSTPAID invoice, account balance, activation charge, or other invoice-like receivable.

PAY-GW calls:

```
POST /api/payments/receive
```

Target module:

```
BIL-01-PAY-01
```

Required normalized payload:

```

{
  "operatorCode": "WIK",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "invoiceId": "INV-2026-00001234",
  "paymentReference": "MPESA-QL91ABCD",
  "paidAmount": 2500.00,
  "currency": "KES",
  "paymentMethod": "MOBILE_MONEY",
  "gatewayProviderCode": "MPESA_KE",
  "gatewayTransactionId": "GTX-20260529-000001",
  "paidAt": "2026-05-29T09:42:00Z"
}

```

invoiceId may be null if the reference maps only to a customer account. In that case, PAY-01 uses its own allocation rules.

7.2 Route To Wallet Topup

Use this route when the payment is meant to increase a PREPAID wallet balance.

PAY-GW calls an internal BIL-05 credit API:

```
POST /api/wallets/{wallet_id}/credit
```

Target module:

```
BIL-05
```

Required normalized payload:

```
{
  "operatorCode": "WIK",
  "walletId": "WAL-200045",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "amount": 1000.00,
  "currency": "KES",
  "creditType": "EXTERNAL_TOPUP",
  "gatewayProviderCode": "MPESA_KE",
  "gatewayReference": "MPESA-QL91EFGH",
  "gatewayTransactionId": "GTX-20260529-000002",
  "paidAt": "2026-05-29T09:45:00Z"
}
```

If the wallet cannot be resolved, PAY-GW stores the transaction as HELD with reason UNKNOWN_WALLET_ALIAS.

7.3 Route Resolution Inputs

PAY-GW may use these fields to resolve the target:

Input	Purpose
Provider transaction id	Idempotency and audit.
Customer entered account reference	Main route to customer, invoice, account, or wallet.
Payer MSISDN	Fallback wallet alias or fraud investigation input.
Paybill/business number	Helps select channel and operator.
Amount/currency	Validation and routing payload.
Provider payment time	Accounting timestamp.
Channel target mode	Tells whether the channel defaults to PAY-01, BIL-05, or auto-route.

8. Status Model

8.1 Gateway Transaction Status

Status	Meaning
RECEIVED	Callback or settlement row has been stored.
VALIDATED	Signature, payload, amount, and provider rules passed.
ROUTING_PENDING	Transaction is ready for routing worker.
ROUTING_RETRY	Last routing attempt failed due to recoverable issue.
APPLIED	Target module accepted and posted the payment/topup.
HELD	Human or configured repair action is required.
REJECTED	Invalid callback or non-recoverable provider payload.
DUPLICATE	Same payment already processed successfully.
DUPLICATE_CONFLICT	Same provider id with conflicting values. Requires review.
REVERSED	Provider sent reversal and internal reversal was completed.
SETTLED	Provider settlement confirms the transaction.
RECONCILED	Provider settlement and internal records match.

8.2 Held Reasons

Reason	Meaning	Typical action
UNKNOWN_REFERENCE	Customer entered reference does not map to known object.	Route manually to customer/account/wallet or refund offline.
UNKNOWN_WALLET_ALIAS	Payer number or alias did not resolve to wallet.	Link alias or route manually.
MULTIPLE_MATCHES	Reference matches multiple customers or wallets.	Select correct target.
DUPLICATE_CONFLICT	Same provider transaction id has different values.	Compare provider record and choose action.
SIGNATURE_FAILED	Provider signature validation failed.	Reject unless provider confirms issue.
TARGET_MODULE_DOWN	PAY-01 or BIL-05 temporarily unavailable.	Retry routing.

Reason	Meaning	Typical action
TARGET_REJECTED	Target module returned business rejection.	Repair source data or hold.

9. Business Rules

Rule ID	Rule	Notes
R-PAYGW-01	Every inbound provider callback must be stored before provider acknowledgement.	Prevent lost payments.
R-PAYGW-02	Every provider integration must define an idempotency key extraction rule.	Usually provider transaction id plus provider code.
R-PAYGW-03	Duplicate callback with same normalized values returns success and does not re-route.	Provider retry safe.
R-PAYGW-04	Duplicate callback with conflicting normalized values is held for operations review.	Do not silently pick one.
R-PAYGW-05	Invalid signature is rejected and never routed.	Store minimal audit when safe.
R-PAYGW-06	Valid payment with unresolved customer/reference is held, not rejected.	Money may be real.
R-PAYGW-07	Routing to PAY-01 or BIL-05 must use internal service authentication.	Public provider traffic never reaches billing modules directly.
R-PAYGW-08	PAY-GW must not update invoice balances or wallet balances directly.	Target modules own ledgers.
R-PAYGW-09	Gateway reconciliation compares provider totals to PAY-GW transactions and target module application status.	Avoid only comparing one side.
R-PAYGW-10	Raw payloads must be redacted before standard logs.	Store sensitive raw payload only encrypted if needed.
R-PAYGW-11	A gateway transaction may have many routing attempts but only one successful target application.	Enforced by unique idempotency.
R-PAYGW-12	Channel display instructions are configuration and must not be hardcoded in frontend apps.	Supports Kenya first and future regions.

10. Data Model

10.1 payment_gateway_provider

Stores each external payment provider integrated with an operator.

Use this table when onboarding or disabling a provider such as M-Pesa Kenya, a bank notification provider, or a card processor.

Column	Type	Purpose	Sample
provider_id	UUID PK	Internal provider id.	prv-001
operator_code	TEXT	Operator/tenant.	WIK
provider_code	TEXT	Stable code used by APIs/events.	MPESA_KE
provider_name	TEXT	Human-readable name.	M-Pesa Kenya
provider_type	TEXT	MOBILE_MONEY, BANK, CARD, RETAIL_AGENT.	MOBILE_MONEY
status	TEXT	ACTIVE, SUSPENDED, RETIRED.	ACTIVE
callback_url_path	TEXT	Public callback route path.	/api/payment-gateways/mpesa/callback
settlement_mode	TEXT	API, SFTP_FILE, MANUAL_UPLOAD, NONE.	API
currency_code	TEXT	Default currency.	KES
config_json	JSONB	Provider-specific non-secret config.	{"paybill":"123456"}
secret_ref	TEXT	Secret manager reference, not secret value.	secret/paygw/mpesa-ke
created_at	TIMESTAMP	Creation timestamp.	2026-05-29T08:00:00Z

Column	Type	Purpose	Sample
updated_at	TIMESTAMP	Last update timestamp.	2026-05-29T08:00:00Z

Recommended indexes:

- Unique (operator_code, provider_code).
- (operator_code, status).

10.2 payment_gateway_channel

Stores customer-visible payment channels under a provider.

Use this table when configuring different payment routes for invoices, wallet topups, or sales activation payments.

Column	Type	Purpose	Sample
channel_id	UUID PK	Internal channel id.	chn-001
provider_id	UUID FK	Provider owning the channel.	prv-001
channel_code	TEXT	Stable channel code.	MPESA_PAYBILL
payment_method	TEXT	Customer payment method.	MOBILE_MONEY
target_mode	TEXT	PAYMENT_APPLICATION, WALLET_TOPUP, AUTO_ROUTE.	AUTO_ROUTE
status	TEXT	ACTIVE, HIDDEN, SUSPENDED.	ACTIVE
display_name	TEXT	Short label for frontend.	M-Pesa Paybill
display_instructions_json	JSONB	Localized instructions for FE apps.	{"en": "Use Paybill 123456"}
routing_priority	INTEGER	Priority if multiple channels match.	10
metadata_json	JSONB	Provider/channel-specific metadata.	{"businessShortCode": "123456"}

Recommended indexes:

- Unique (provider_id, channel_code).
- (status, target_mode).

10.3 gateway_callback_event

Stores each raw inbound provider callback or provider notification.

Use this table for audit, duplicate detection, replay, and troubleshooting.

Column	Type	Purpose	Sample
callback_event_id	UUID PK	Internal callback event id.	cbe-001
provider_id	UUID FK	Provider that sent the callback.	prv-001
provider_event_id	TEXT	Provider event id if supplied.	evt-998877
provider_transaction_id	TEXT	Provider payment transaction id.	MPESA-QL91ABCD
idempotency_key	TEXT	Normalized dedupe key.	MPESA_KE:MPESA-QL91ABCD
received_at	TIMESTAMP	Time SOPHIX received callback.	2026-05-29T09:42:05Z
signature_status	TEXT	VALID, INVALID, NOT_CONFIGURED.	VALID
parse_status	TEXT	PARSED, MALFORMED, UNSUPPORTED.	PARSED
http_status_returned	INTEGER	HTTP status returned to provider.	200
raw_payload_redacted	JSONB	Redacted payload for support.	{"amount": "2500.00"}
raw_payload_secret_ref	TEXT NULL	Optional encrypted blob reference.	s3://...
created_at	TIMESTAMP	Insert timestamp.	2026-05-29T09:42:05Z

Recommended indexes:

- Unique (provider_id, idempotency_key) when provider idempotency is available.
- (provider_id, received_at).

10.4 gateway_transaction

Stores the normalized payment transaction known by SOPHIX.

Use this table as the operational source of truth for gateway status, routing, reconciliation, and repair.

Column	Type	Purpose	Sample
gateway_txn_id	UUID PK	Internal transaction id.	gtx-001
operator_code	TEXT	Operator/tenant.	WIK
provider_id	UUID FK	Payment provider.	prv-001
channel_id	UUID FK	Payment channel.	chn-001
provider_transaction_id	TEXT	Provider transaction id.	MPESA-QL91ABCD
payment_reference	TEXT	Customer-entered reference or invoice/account ref.	ACC-900045
payer_msisdn	TEXT NULL	Payer phone number where applicable.	254700111222
amount	DECIMAL(18,2)	Paid amount.	2500.00
currency_code	TEXT	Currency.	KES
paid_at	TIMESTAMP	Provider payment time.	2026-05-29T09:42:00Z
status	TEXT	Gateway status.	APPLIED
hold_reason	TEXT NULL	Reason if held.	UNKNOWN_REFERENCE
target_type	TEXT	INVOICE, ACCOUNT, WALLET, CUSTOMER, UNKNOWN.	ACCOUNT
target_id	TEXT NULL	Resolved target id.	ACC-900045
customer_id	TEXT NULL	Resolved customer id.	CUS-100045
account_id	TEXT NULL	Resolved billing account id.	ACC-900045
subscription_id	TEXT NULL	Resolved subscription id when applicable.	SUB-700045
wallet_id	TEXT NULL	Resolved wallet id when applicable.	WAL-200045
application_module	TEXT NULL	BIL-01-PAY-01 or BIL-05.	BIL-01-PAY-01
application_reference_id	TEXT NULL	Payment id or wallet transaction id returned by target module.	PAY-12345
last_error_code	TEXT NULL	Last routing or validation error.	TARGET_TIMEOUT
created_at	TIMESTAMP	Creation timestamp.	2026-05-29T09:42:05Z
updated_at	TIMESTAMP	Last update timestamp.	2026-05-29T09:42:10Z

Recommended indexes:

- Unique (provider_id, provider_transaction_id).
- (operator_code, status, paid_at).
- (customer_id, paid_at).
- (payment_reference, paid_at).

10.5 gateway_routing_attempt

Stores every attempt to route a gateway transaction into an internal billing module.

Use this table to debug whether PAY-01 or BIL-05 accepted or rejected a transaction.

Column	Type	Purpose	Sample
attempt_id	UUID PK	Internal attempt id.	gra-001
gateway_txn_id	UUID FK	Gateway transaction.	gtx-001
attempt_no	INTEGER	Attempt sequence.	1
target_module	TEXT	Module called.	BIL-01-PAY-01
target_endpoint	TEXT	Internal endpoint called.	/api/payments/receive
status	TEXT	SUCCESS, FAILED_RETRYABLE, FAILED_FINAL.	SUCCESS

Column	Type	Purpose	Sample
request_payload_redacted	JSONB	Redacted internal request.	{"paidAmount":2500}
response_status	INTEGER	HTTP status from target.	201
response_payload_redacted	JSONB	Redacted response.	{"paymentId":"PAY-12345"}
error_code	TEXT NULL	Error code if failed.	TARGET_TIMEOUT
created_at	TIMESTAMP	Attempt time.	2026-05-29T09:42:08Z

Recommended indexes:

- (gateway_txn_id, attempt_no).
- (status, created_at).

10.6 gateway_reconciliation_run

Stores one provider settlement comparison run.

Use this table for daily finance reconciliation and operational dashboards.

Column	Type	Purpose	Sample
run_id	UUID PK	Reconciliation run id.	rec-001
provider_id	UUID FK	Provider reconciled.	prv-001
settlement_date	DATE	Provider settlement business date.	2026-05-29
status	TEXT	RUNNING, COMPLETED, FAILED, PARTIAL.	COMPLETED
provider_total_amount	DECIMAL(18,2)	Provider settlement total.	500000.00
sophix_total_amount	DECIMAL(18,2)	Matching SOPHIX gateway total.	499000.00
provider_txn_count	INTEGER	Count from provider.	220
sophix_txn_count	INTEGER	Count from SOPHIX.	219
discrepancy_count	INTEGER	Number of mismatch rows.	1
started_at	TIMESTAMP	Run start.	2026-05-30T01:00:00Z
completed_at	TIMESTAMP NULL	Run completion.	2026-05-30T01:05:00Z

Recommended indexes:

- Unique (provider_id, settlement_date).
- (status, settlement_date).

10.7 gateway_reconciliation_item

Stores one discrepancy or confirmation row from settlement reconciliation.

Use this table when operations needs to resolve missing, mismatched, or duplicate provider records.

Column	Type	Purpose	Sample
item_id	UUID PK	Reconciliation item id.	rci-001
run_id	UUID FK	Reconciliation run.	rec-001
provider_transaction_id	TEXT	Provider transaction id.	MPESA-QL91ZZZZ
gateway_txn_id	UUID NULL	Matching gateway transaction if found.	gtx-001
discrepancy_type	TEXT	MISSING_IN_BSS, MISSING_IN_PROVIDER, AMOUNT_MISMATCH, STATUS_MISMATCH, DUPLICATE.	MISSING_IN_BSS
provider_amount	DECIMAL(18,2) NULL	Amount from provider.	1000.00
sophix_amount	DECIMAL(18,2) NULL	Amount from SOPHIX.	NULL
status	TEXT	OPEN, RESOLVED, IGNORED, ESCALATED.	OPEN
resolution_notes	TEXT NULL	Operations resolution notes.	Callback replay requested
created_at	TIMESTAMP	Creation timestamp.	2026-05-30T01:05:00Z

Column	Type	Purpose	Sample
resolved_at	TIMESTAMP NULL	Resolution timestamp.	NULL

Recommended indexes:

- (run_id, status).
- (provider_transaction_id).

10.8 gateway_pending_resolution

Stores the current repair queue for held gateway transactions.

Use this table to drive Backoffice payment operations screens.

Column	Type	Purpose	Sample
pending_id	UUID PK	Pending resolution id.	gpr-001
gateway_txn_id	UUID FK	Gateway transaction needing action.	gtx-002
reason_code	TEXT	Held reason.	UNKNOWN_REFERENCE
assigned_to_user_id	TEXT NULL	Backoffice user assigned to resolve.	usr-902
status	TEXT	OPEN, IN_REVIEW, RESOLVED, CANCELLED.	OPEN
resolution_action	TEXT NULL	ROUTE_TO_PAYMENT, ROUTE_TO_WALLET, MARK_DUPLICATE, IGNORE, OFFLINE_REFUND.	ROUTE_TO_PAYMENT
resolution_payload	JSONB NULL	Target selected by operations.	{"accountId": "ACC-900045"}
created_at	TIMESTAMP	Creation timestamp.	2026-05-29T09:45:00Z
resolved_at	TIMESTAMP NULL	Resolution timestamp.	NULL

Recommended indexes:

- (status, reason_code, created_at).
- (assigned_to_user_id, status).

11. API Contract

11.1 Public Provider Callback APIs

These endpoints are internet-facing through API gateway/WAF controls.

```
POST /api/payment-gateways/mpesa/callback
POST /api/payment-gateways/bank/callback
POST /api/payment-gateways/card/callback
POST /api/payment-gateways/retail-agent/callback
```

Each provider adapter owns provider-specific parsing but must output the same normalized transaction shape.

11.2 Internal Configuration APIs

```
GET /api/payment-gateways/providers
POST /api/payment-gateways/providers
GET /api/payment-gateways/providers/{provider_id}
PATCH /api/payment-gateways/providers/{provider_id}

GET /api/payment-gateways/channels
POST /api/payment-gateways/channels
GET /api/payment-gateways/channels/{channel_id}
PATCH /api/payment-gateways/channels/{channel_id}
```

Configuration APIs require Backoffice admin permissions from EM-CFG-03.

11.3 Operations APIs

```
GET /api/payment-gateways/transactions
GET /api/payment-gateways/transactions/{gateway_txn_id}
POST /api/payment-gateways/transactions/{gateway_txn_id}/retry-routing
```

```

GET /api/payment-gateways/pending-resolutions
POST /api/payment-gateways/pending-resolutions/{pending_id}/route-to-payment
POST /api/payment-gateways/pending-resolutions/{pending_id}/route-to-wallet
POST /api/payment-gateways/pending-resolutions/{pending_id}/mark-duplicate
POST /api/payment-gateways/pending-resolutions/{pending_id}/ignore

```

11.4 Reconciliation APIs

```

GET /api/payment-gateways/reconciliation-runs
POST /api/payment-gateways/reconciliation-runs
GET /api/payment-gateways/reconciliation-runs/{run_id}
GET /api/payment-gateways/reconciliation-items?runId={run_id}
POST /api/payment-gateways/reconciliation-items/{item_id}/resolve

```

12. Events

12.1 sophix.paygw.payment.received

Emitted when a valid callback or settlement row creates a gateway transaction.

```

{
  "eventType": "GatewayPaymentReceived",
  "gatewayTxnId": "GTX-20260529-000001",
  "operatorCode": "WIK",
  "providerCode": "MPESA_KE",
  "providerTransactionId": "MPESA-QL91ABCD",
  "amount": 2500.00,
  "currency": "KES",
  "paymentReference": "ACC-900045",
  "receivedAt": "2026-05-29T09:42:05Z"
}

```

12.2 sophix.paygw.payment.held

Emitted when a gateway transaction cannot be automatically routed.

```

{
  "eventType": "GatewayPaymentHeld",
  "gatewayTxnId": "GTX-20260529-000003",
  "operatorCode": "WIK",
  "providerCode": "MPESA_KE",
  "holdReason": "UNKNOWN_REFERENCE",
  "amount": 1000.00,
  "currency": "KES"
}

```

12.3 sophix.paygw.payment.applied

Emitted after target module accepts the payment/topup.

```

{
  "eventType": "GatewayPaymentApplied",
  "gatewayTxnId": "GTX-20260529-000001",
  "applicationModule": "BIL-01-PAY-01",
  "applicationReferenceId": "PAY-12345",
  "customerId": "CUS-100045",
  "amount": 2500.00,
  "currency": "KES"
}

```

12.4 sophix.paygw.reconciliation.discrepancycreated

Emitted when reconciliation creates at least one open discrepancy.

```

{
  "eventType": "GatewayReconciliationDiscrepancyCreated",
  "runId": "REC-20260530-MPESA",
  "providerCode": "MPESA_KE",
  "settlementDate": "2026-05-29",
  "discrepancyCount": 3
}

```

13. Frontend Usage

13.1 FE-APP-01 Backoffice

Backoffice uses PAY-GW-01 for:

- Payment gateway transaction search.
- Held payment repair queue.
- Provider/channel configuration.
- Gateway reconciliation run and discrepancy resolution.
- Retry routing action.

Backoffice does not use PAY-GW-01 to apply payments directly unless resolving a held gateway transaction. Normal manual offline payments remain in PAY-01.

13.2 FE-APP-02 Outdoor Sales App

Outdoor sales app uses PAY-GW-01 channel configuration to show payment instructions during sales or activation flows.

It should not hardcode Kenya-specific labels. It should request active channel instructions for the operator and context.

13.3 FE-APP-04 Customer Self Care

Self-care app uses PAY-GW-01 channel configuration to show payment instructions and available methods.

If STK push or payment initiation is later implemented, PAY-GW-01 should own the provider initiation endpoint and still route successful provider confirmation back to PAY-01 or BIL-05.

14. Security Requirements

Area	Requirement
Public callback exposure	Only PAY-GW public endpoints are exposed to providers. BIL-01 and BIL-05 remain internal.
Signature validation	Each provider adapter must verify provider signature, HMAC, mTLS, or configured equivalent.
IP allowlist	Use where provider supports stable callback IP ranges. Do not rely on IP allowlist alone.
Secrets	Store provider credentials in secret manager, never in DB config JSON or code.
Payload redaction	Standard logs must not contain full phone numbers, card data, tokens, or provider credentials.
Replay protection	Idempotency key and timestamp windows must be enforced per provider.
RBAC	Repair, ignore, mark duplicate, and provider configuration actions require Backoffice permissions.
Audit	Every manual repair action must store user id, before/after status, reason, and timestamp.

15. Infra Recommendation

15.1 Medium Budget Deployment

PAY-GW-01 can run as two stateless API pods plus one or more worker pods.

Recommended starting point:

Component	Recommendation
API pods	2 pods, 1 vCPU, 1.5-2 GB RAM each.
Worker pods	1-2 pods, 1 vCPU, 2 GB RAM each.
DB	Shared billing PostgreSQL cluster, dedicated schema paygw.
Kafka	Shared platform Kafka topics.
Redis	Shared Redis for short-lived provider nonce/replay cache only. Do not use as source of truth.

Component	Recommendation
Object storage	Optional encrypted storage for full raw payload archives if legally required.

15.2 Recommended Partitions

Partition high-volume tables by month:

- gateway_callback_event
- gateway_transaction
- gateway_routing_attempt

Retention:

- Callback events: 24 months online, archive after that according to finance/audit policy.
- Gateway transactions: 7-10 years if used for finance audit, or move older data to reporting/archive.
- Routing attempts: 24 months online, summarized after that.

15.3 OS Recommendation

Use Ubuntu Server 24.04 LTS for VPS/bare-metal style deployments, or hardened container images based on Debian/Ubuntu slim for Kubernetes.

Minimum OS hardening:

- Automatic security patching.
- NTP/time synchronization.
- No public SSH to application nodes where possible.
- Host firewall allowing only load balancer and internal network traffic.
- Log forwarding enabled.

16. AWS Deployment Recommendation

For AWS, deploy PAY-GW-01 with a separate internet-facing path but keep the service private.

Recommended medium deployment:

AWS component	Usage
AWS WAF	Protect public callback paths with rate limits and provider-specific rules.
ALB	Public entry point for provider callbacks.
ECS/EKS	Run PAY-GW API and worker containers.
RDS PostgreSQL	Store PAY-GW schema in the billing DB cluster or a dedicated finance DB if required.
MSK or managed Kafka equivalent	Gateway events and async routing jobs.
Secrets Manager	Provider credentials, certificates, API keys.
S3 with KMS	Optional encrypted raw payload archive and settlement files.
CloudWatch	Metrics, alarms, structured logs.

Operational alarms:

- Callback 5xx rate above threshold.
- Invalid signature spike.
- Routing retry backlog.
- Held payment count above threshold.
- Reconciliation discrepancy count above threshold.
- No callbacks received during expected business hours for active provider.

17. Testing Requirements

17.1 Unit Tests

- Provider parser normalizes valid callback payloads.
- Invalid signature is rejected.
- Duplicate callback returns existing transaction.
- Duplicate conflict creates held transaction.
- Redaction removes sensitive fields.
- Routing payload mapping to PAY-01 is correct.
- Routing payload mapping to BIL-05 is correct.

17.2 Integration Tests

- Provider callback creates gateway transaction and returns 200 after durable store.
- Routing worker calls PAY-01 once for POSTPAID payment.
- Routing worker calls BIL-05 once for wallet topup.
- PAY-01 temporary outage triggers retry without duplicate application.
- BIL-05 rejection creates held resolution.
- Reconciliation run creates expected discrepancy items.

17.3 End To End Tests

- Customer pays invoice via provider callback, invoice becomes paid through PAY-01.
- Customer tops up PREPAID wallet, wallet balance increases through BIL-05.
- Unknown reference payment appears in Backoffice held queue.
- Operations routes held payment to account, transaction becomes applied.
- Provider settlement file with one missing row creates an open reconciliation item.

18. Implementation Notes

1. Implement provider adapters behind a common interface:

```
PaymentProviderAdapter
  verifySignature(request)
  parseCallback(request)
  buildIdempotencyKey(parsedPayload)
  parseSettlementRow(row)
```

2. Keep provider-specific code isolated from core routing logic.
3. Store callbacks and transactions in one database transaction where possible.
4. Return provider acknowledgement only after durable write.
5. Make routing workers idempotent by checking `gateway_transaction.application_reference_id` before calling target modules.
6. Use target module idempotency keys:
 - PAY-01: `paymentReference = provider_transaction_id` or stable gateway payment reference.
 - BIL-05: `gatewayReference = provider_transaction_id`.
7. Do not route invalid signatures.
8. Do not reject valid but unresolved money. Hold it for operations.
9. The frontend must read channel display configuration from PAY-GW-01, not hardcode M-Pesa or bank text.

19. Open Questions

Question	Recommendation
Which exact provider APIs are confirmed for Kenya launch?	Start with generic M-Pesa-style callback adapter and finalize exact parser when provider contract is available.
Is STK push initiation needed at launch or only passive payment callbacks?	Design supports both, but implement passive callback first if budget is limited.
Should PAY-GW own outbound refunds/payouts?	Keep inbound gateway and outbound payout separate, but reuse provider catalog where practical.
Should raw payloads be archived long-term?	Ask finance/compliance. Default to redacted DB payload plus optional encrypted archive.

20. Definition Of Done

PAY-GW-01 is complete when:

- Provider and channel configuration APIs are implemented.
- At least one provider callback adapter is implemented.
- Callback events and gateway transactions are durably stored.
- Duplicate and duplicate-conflict handling is tested.
- Routing to PAY-01 and BIL-05 works with idempotency.
- Held payment repair queue works in Backoffice.
- Daily reconciliation run can import or fetch provider settlement data.
- Security controls for callback endpoints are active.
- Metrics and alerts are configured.
- FE apps can read active payment channel instructions by operator and context.